PLUTO BANKERS

Banking System — Documentation Report

Group Assignment | Total Marks: 300

Field	Details
Assignment Title	Banking System — Java & SQL Group Assignment
Course	Advanced Programming & Database Systems
Lecturer	Mr Owami
Student Name And Student Number	Shaun Khalira MA977363
Student Name And Student Number	Funwi Checy Che 1195270
Student Name And Student Number	Jordan Olivier
Submission Date	27 May 2026
GitHub Repository	https://github.com/Nameless-8-Kidd/PLUTO-BANKERS-JAVA-BANKING-SYSTEM-.git

TABLE OF CONTENTS

Section	Title	Page
1	Introduction	3
2	System Overview	4
3	Database Design	6
4	Application Features Documentation	11
5	User Manual	21
6	Challenges and Reflections	24
Appendix A	Full SQL Schema & Queries	25
Appendix B	Java Source Code Reference	31

1. INTRODUCTION

1.1 Purpose and Scope

This document presents the full technical documentation for ***OUR PLUTO BANKERS Banking System***, a Java-based command-line application backed by a relational MySQL database. The system was developed as a group assignment to demonstrate practical skills in object-oriented programming, relational database design, and software documentation.

The system provides two categories of users — clients and employees — with access to a range of core banking operations including account management, transaction processing, balance enquiries, and customer record management.

1.2 Background and Motivation

Modern banking institutions depend on reliable, structured software systems to manage customer accounts and process financial transactions securely and accurately. The ***PLUTO BANKERS*** system is built to simulate the core functionality of a real-world banking application, demonstrating how software engineering principles apply in a financial context.

The motivation behind this project was to gain hands-on experience in designing and building a complete application that integrates a front-end user interface (command-line), back-end business logic (Java), and a persistent data store (MySQL). Understanding how these three layers interact is a fundamental skill for any software developer.

1.3 Objectives

1. Design and implement a fully functional Java banking application with employee and client portals.
2. Create a normalised relational MySQL database with at least five related tables.
3. Implement all core banking operations: registration, login, deposits, withdrawals, and transfers.
4. Apply input validation and error handling to prevent invalid operations.
5. Document the system thoroughly in accordance with the assignment requirements.
6. Commit all source code and SQL files to a public GitHub repository.

2. SYSTEM OVERVIEW

2.1 High-Level Description

PLUTO BANKERS is a menu-driven, command-line Java application that simulates the core operations of a banking system. It supports two distinct user roles: employees who manage client accounts and records, and clients who perform personal banking transactions. All data is stored in parallel arrays in memory (reflecting the Java implementation) with a corresponding relational MySQL database capturing the same structure for persistence.

The application launches into a main menu from which users navigate to role-specific functions. Sessions are tracked via static login index variables, ensuring that operations performed during a session are attributed to the correct user.

2.2 Core Functionalities

Client Portal

- **Check Balance:** View current account balance with options to send money, withdraw, or deposit.
- **Deposit:** Add funds to the account; balance is updated in memory and transaction recorded.
- **Withdraw:** Remove funds from the account with confirmation; validates against available balance.
- **Transfer Money:** Send funds to another account by number; validates recipient, deducts from sender, credits recipient.
- **Update Credentials:** Update password, address, email, or phone number in-session.
- **View Transaction History:** Display a full log of all deposits, withdrawals, and transfers for the session.

Employee Portal

- **View All Clients:** Display a complete list of all registered clients and their account details.
- **View Client Profile:** Look up a specific client by account number; shows all personal and account information.
- **Edit Client Details:** Modify client password, address, email, or phone number using account number lookup.
- **View Transaction History:** Access the transaction history of any client by account number.
- **Close Account:** Mark a client account as CLOSED; prevents future logins for that client.

2.3 Technologies and Tools Used

Technology / Tool	Version / Details	Purpose
Java (JDK)	Java 17 (or JDK 21)	Primary programming language; application logic

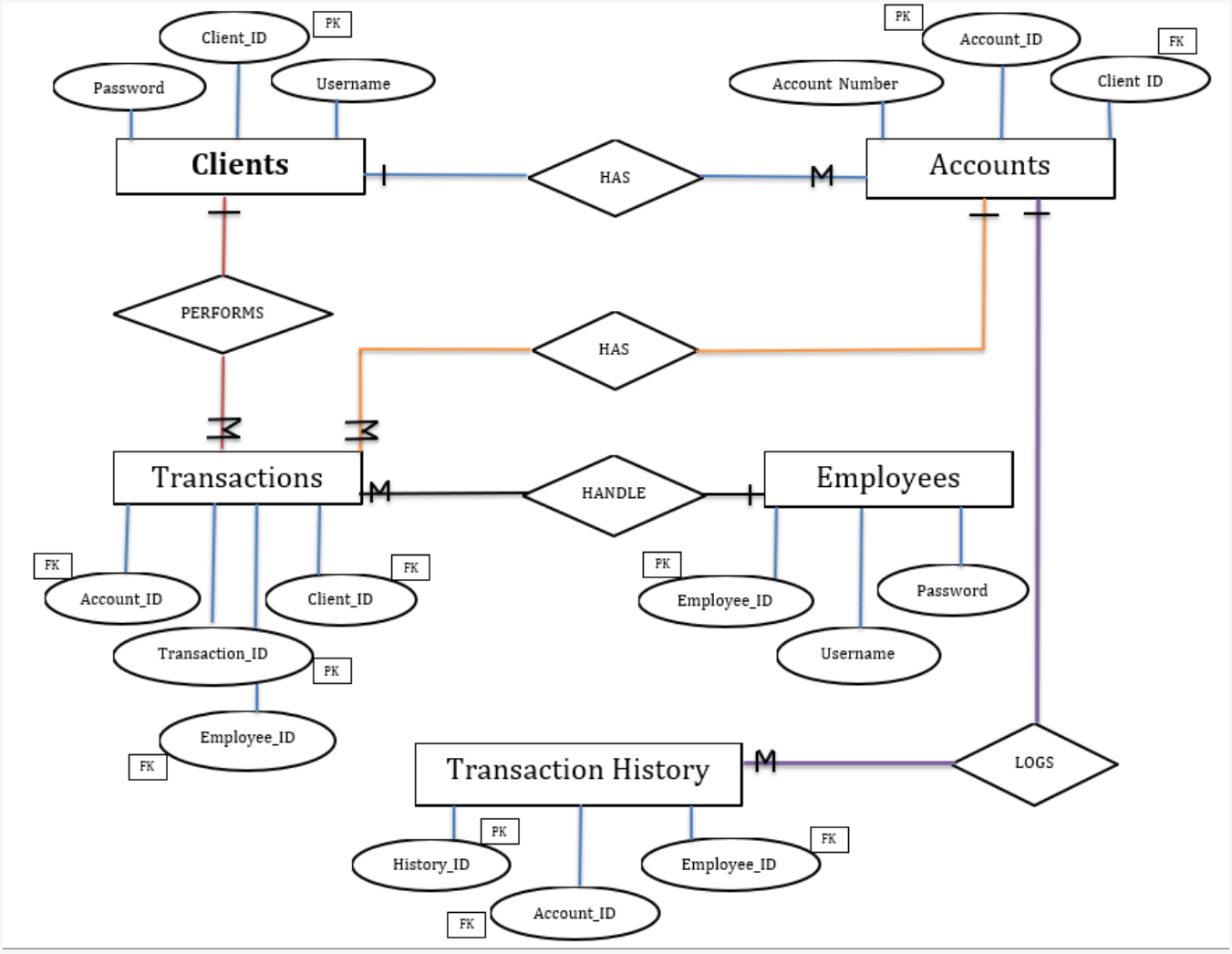
Apache NetBeans IDE	NetBeans 21	Development environment; code writing and testing
MySQL	MySQL 8.0+	Relational database management system
MySQL Workbench	8.0+	Database design, query execution, and ERD creation
GitHub	git / GitHub.com	Version control and code submission repository
java.util.Scanner	Standard Library	Console input handling
java.util.Random	Standard Library	Random account number and PIN generation
java.time.LocalTime	Standard Library	Timestamping transactions

3. DATABASE DESIGN

3.1 Entity-Relationship Diagram (ERD)

The ERD below illustrates all five tables, their attributes, primary keys, and foreign key relationships.

PLUTO BANKERS ENTITY-RELATIONSHIP-DIAGRAM



3.2 Table Descriptions

Table 1: clients

Stores all personal information for registered bank clients. Each client is uniquely identified by a client_id. This table is the central entity in the schema — all other tables reference it directly or indirectly.

Column	Data Type	Constraints	Description

client_id	VARCHAR(50)	PRIMARY KEY	Unique client identifier (e.g. CUS01)
username	VARCHAR(50)	UNIQUE, NOT NULL	Login username
password	VARCHAR(100)	NOT NULL	Login password
full_name	VARCHAR(100)	NOT NULL	Client's full name
surname	VARCHAR(100)	NOT NULL	Client's surname
email	VARCHAR(100)	UNIQUE	Email address
phone	VARCHAR(20)	—	Contact phone number
address	TEXT	—	Residential address
gender	VARCHAR(10)	—	Gender (Male / Female)
status	VARCHAR(20)	DEFAULT 'ACTIVE'	Account status: ACTIVE or CLOSED

Table 2: accounts

Stores bank account details linked to each client. A client may have one account. This table holds the financial data including balance, account type, and PIN.

Column	Data Type	Constraints	Description
account_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique account record identifier
client_id	VARCHAR(50)	NOT NULL, FOREIGN KEY → clients	Links account to a client
account_number	BIGINT	UNIQUE, NOT NULL	Publicly visible account number
account_pin	INT	NOT NULL	4-digit security PIN
account_type	VARCHAR(50)	—	Type: Savings, Checking, Salary, Joint
balance	DECIMAL(12,2)	DEFAULT 0.00	Current account balance

Table 3: employees

Stores login credentials and names for bank employees. Employees do not have client accounts — they manage client data via the employee portal.

Column	Data Type	Constraints	Description
employee_id	VARCHAR(50)	PRIMARY KEY	Unique employee identifier (e.g. EMP01)
username	VARCHAR(50)	UNIQUE, NOT NULL	Login username
password	VARCHAR(100)	NOT NULL	Login password
full_name	VARCHAR(100)	—	Employee's full name

Table 4: transactions

Records every financial transaction performed in the system. Each row represents one transaction event — deposit, withdrawal, or transfer — and is linked to the account, client, and the employee who supervised it.

Column	Data Type	Constraints	Description
transaction_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique transaction identifier
account_id	INT	NOT NULL, FK → accounts	Account the transaction belongs to
employee_id	VARCHAR(50)	FK → employees	Employee who processed the transaction
client_id	VARCHAR(50)	FK → clients	Client involved in the transaction
transaction_type	VARCHAR(50)	—	DEPOSIT / WITHDRAWAL / TRANSFER SENT / RECEIVED
amount	DECIMAL(12,2)	—	Transaction amount in Rands
reference_note	VARCHAR(255)	—	User-provided reference or description
transaction_date	DATE	DEFAULT CURRENT_DATE()	Date the transaction occurred
Time	TIME	DEFAULT CURRENT_TIMESTAMP	Time the transaction occurred

Table 5: transaction_history

Stores a human-readable activity log per account. Unlike the transactions table (which stores structured data), this table stores formatted string entries matching the output shown to users in the application.

Column	Data Type	Constraints	Description
history_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique history record identifier
account_number	BIGINT	FK → accounts(account_number)	Account this activity belongs to
activity	TEXT	—	Formatted activity string (e.g. DEPOSIT | Amount: R5000.00 | ...)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Automatic timestamp of the activity
employee_id	VARCHAR(50)	FK → employees	Employee who performed or oversaw the activity

3.3 Primary and Foreign Key Justification

Every table uses a purpose-specific primary key rather than a generic integer where possible:

- **clients.client_id (VARCHAR):** Chosen as a descriptive identifier (e.g. CUS01) to make data readable in joins and reports without requiring additional lookups.
- **accounts.account_id (INT AUTO_INCREMENT):** A surrogate key is appropriate here as accounts do not have a natural single-field identifier. account_number is UNIQUE but BIGINT — using account_id keeps foreign key references efficient.
- **employees.employee_id (VARCHAR):** Same reasoning as client_id — descriptive format (EMP01) improves readability in transaction records.
- **transactions.transaction_id (INT AUTO_INCREMENT):** Transactions have no natural key; a surrogate auto-increment is the standard choice.
- **transaction_history.history_id (INT AUTO_INCREMENT):** Same as transactions — each log entry is unique only by its sequential identifier.

Foreign key choices enforce referential integrity across the schema:

- **accounts.client_id → clients.client_id:** Ensures no account can exist without a valid client. Prevents orphaned account records.
- **transactions.account_id → accounts.account_id:** Ensures every transaction is tied to a real account.
- **transactions.employee_id → employees.employee_id:** Ensures the employee reference is valid; allows NULL for client-self-service transactions.
- **transactions.client_id → clients.client_id:** Provides a direct link from transaction to client for reporting without joining through accounts.
- **transaction_history.account_number → accounts.account_number:** Links history entries to the specific account number displayed to users.
- **transaction_history.employee_id → employees.employee_id:** Tracks which employee supervised the activity for audit purposes.

3.4 Normalisation

The database schema achieves Third Normal Form (3NF):

- **First Normal Form (1NF):** All tables have a primary key, and all column values are atomic (no multi-valued attributes or repeating groups). Each row is uniquely identifiable.
- **Second Normal Form (2NF):** All non-key attributes are fully functionally dependent on the entire primary key. In composite-key-free tables (all tables here use single-column PKs), 2NF is automatically satisfied if 1NF holds.
- **Third Normal Form (3NF):** No transitive dependencies exist. For example, client contact details (email, phone, address) depend only on client_id, not on any non-key attribute. Account balance depends only on account_id.

> **NORMALISATION NOTE:** The transactions table stores both client_id and account_id. Although client_id can be derived via account_id → accounts → client_id, the direct foreign key

> is intentional — it simplifies reporting queries (e.g. SELECT 5) by avoiding an additional JOIN, and is a common denormalisation trade-off in transaction systems.

4. APPLICATION FEATURES DOCUMENTATION

4.1 Feature List with Descriptions

Feature	User Role	Description
Main Menu	All	Entry point. Routes to Employee Login, Client Login, Register New Client, or Exiting the ***PLUTO BANKERS*** System.
Client Login	Client	Authenticates client by username and password. Checks account is ACTIVE before granting access.
Employee Login	Employee	Authenticates employee by username and password. Sets loggedInEmployee for audit tracking.
Register New Client	Public	Collects full registration details, validates username uniqueness, generates random account number and PIN, saves to all arrays.
Check Balance	Client	Displays current balance from accountBalance array. Offers navigation to deposit, withdraw, or transfer.
Deposit	Client	Accepts a positive deposit amount, adds to accountBalance, records formatted entry in transactionHistory with timestamp.
Withdraw	Client	Accepts amount; validates against available balance; deducts from accountBalance; records to transactionHistory.
Transfer Money	Client	Accepts amount, recipient account number, and reference; validates recipient exists; deducts from sender; credits recipient; records for both parties.
Update Credentials	Client	Allows in-session update of password, address, email, or phone number directly in the corresponding array.
View Transaction History	Client / Employee	Displays the formatted transactionHistory string for the target account, or 'No Transactions Available' if empty.
View All Clients	Employee	Iterates all client arrays and prints a complete formatted record for each client.
View Client Profile	Employee	Looks up a client by account number and displays their full personal and account profile.
Edit Client Details	Employee	Searches by account number; allows changing password, address, email, or phone for any client.
Close Account	Employee	Sets status to CLOSED for a target account; blocked accounts cannot log in; logs acting employee.

4.2 Screenshots — Main Menu

```
===== WELCOME TO PLUTO BANKERS =====
1. Employee Login
2. Client Login
3. Register Client
4. Exit
Choose Option:
```

4.3 Screenshots — Client Login & Main Menu

```
Choose Option: 2
===== CLIENT LOGIN =====
Enter Username: bob
Enter Password: bob1
Login Successful! Welcome John Mark
===== Welcome Back John Mark  Main Menu =====
1. Check Balance
2. Deposit
3. Withdraw
4. Transfer Money
5. Update Credentials
6. Transaction History
7. Log out
8. Exit
Choose an option:
```

4.4 Screenshots — Deposit

```
===== Deposit Money =====
Deposit into:
1. Savings Account
2. Return Home
Choose an option: 1
Enter Amount: 120
Deposit Successful!
---------------------------------------
Amount: R120,00
New Account Balance: R20154,00
---------------------------------------
1. Return Home
2. Exit
Choose an option:
```

4.5 Screenshots — Withdrawal

```
===== Withdraw =====
Enter Amount: 1000
Please confirm request:
Amount: R1000,00
Withdrawal from Savings Account
1. Proceed
2. Cancel
Choose an option: 1
Withdrawal Successful!
---------------------------------------
Amount: R1000,00
New Balance: R19154,00
---------------------------------------
1. Return Home
2. Exit
Choose an option:
```

4.6 Screenshots — Transfer Money

```
===== Transfer Money =====
Enter Amount: 345
Enter Recipient's Name: Dave
Enter Recipient's Account Number: 8493733
Reference: Salary Amount For The Day
--------------------------------------
Please Confirm Transfer Details
To: Hazard Disman
Account Number: 8493733
Amount: R345,00
Ref: Salary Amount For The Day
--------------------------------------
1. Proceed
2. Cancel
Choose an option: 1
Processing...    DONE
Transfer Successful!
--------------------------------------
To: Hazard Disman
Account Number: 8493733
Amount: R345,00
Ref: Salary Amount For The Day
New Balance: R18809,00
--------------------------------------
1. Return Home
2. Exit
Choose an option:
```

4.7 Screenshots — Transaction History

```
===== TRANSACTION HISTORY =====
Client: John Mark
Account Number: 9343343
------------------------------
DEPOSIT | Amount: R120,00 | New Balance: R20154,00 | Time: 19:57:25
WITHDRAWAL | Amount: R1000,00 | New Balance: R19154,00 | Time: 19:57:25
TRANSFER SENT | To: Hazard Disman (Acc: 8493733) | Amount: R345,00 | Ref: Salary Amount For The Day | New Balance: R18809,00
 | Time: 19:57:25
```

4.8 Screenshots — Employee Portal

```
===== EMPLOYEE LOGIN =====
Enter Username: mike
Enter Password: emp1
Employee Login Successful! Welcome mike
===== EMPLOYEE MENU =====
1. View All Clients
2. View Client Profile
3. Edit Client Details
4. View Transaction History
5. Close Account
6. Logout
Choose Option:
```

```
===== ALL CLIENTS =====
--------------------------------
Client ID       : CUS01
Full Name       : John Mark
Username        : bob
Email           : john@gmail.com
Phone Number    : 0673459873
Address         : 21 mooi street JHB
Account Number  : 9343343
Account Type    : Savings Account
Status          : ACTIVE
Balance         : R18809,00
--------------------------------
Client ID       : CUS02
Full Name       : Paul Mcman
Username        : peter
Email           : paul@gmail.com
Phone Number    : 0663452363
Address         : 101 good street PTA
Account Number  : 4847433
Account Type    : Checking Account
Status          : ACTIVE
Balance         : R54334,00
--------------------------------
```

```
--------------------------------
Client ID      : CUS03
Full Name      : Sandra Moore
Username       : josh
Email          : sandra@gmail.com
Phone Number   : 0654231200
Address        : 211 sandom street JHB
Account Number : 2987654
Account Type   : Salary Account
Status         : ACTIVE
Balance        : R6454,00
--------------------------------
Client ID      : CUS04
Full Name      : Peter Zulu
Username       : sam
Email          : peter@gmail.com
Phone Number   : 0697653473
Address        : 301 nitye street JHB
Account Number : 3043433
Account Type   : Joint Account
Status         : CLOSED
Balance        : R8445,00
--------------------------------
```

```
--------------------------------
Client ID       : CUS05
Full Name       : Hazard Disman
Username        : jack
Email           : hazard@gmail.com
Phone Number    : 0634568765
Address         : 91 mooi street CPT
Account Number : 8493733
Account Type    : Savings Account
Status          : ACTIVE
Balance         : R9889,00
===== EMPLOYEE MENU =====
1. View All Clients
2. View Client Profile
3. Edit Client Details
4. View Transaction History
5. Close Account
6. Logout
Choose Option:
```

```
===== EMPLOYEE MENU =====
1. View All Clients
2. View Client Profile
3. Edit Client Details
4. View Transaction History
5. Close Account
6. Logout
Choose Option: 5
Enter Account Number: 2987654
mike successfully closed Sandra Moore's account.
```

4.9 Screenshots — Register New Client

```
===== CLIENT REGISTRATION =====
Enter Username: ten
Enter Full Name: Dave Blue
Enter Surname: Blue
Enter Email: ten34@gmail.com
Enter Phone Number: 4567658453
Enter Address: 67 Mail Road ST
Enter Gender (Male/Female): male
Enter Password: blue3
Choose Account Type:
1. Savings Account
2. Checking Account
3. Salary Account
Choose: 3
---------------------------------------
Account Successfully Created!
Account Number : 3205656
Account PIN    : 9213
Account Type   : Salary Account
---------------------------------------
```

4.10 Input / Output Descriptions

Feature	Inputs Accepted	Output Produced
Client Login	Username (String), Password (String)	'Login Successful! Welcome [name]' or 'Invalid Login Details!'
Deposit	Menu choice (int 1-2), Amount (double > 0)	Updated balance displayed; transaction appended to history
Withdraw	Amount (double > 0 and ≤ balance)	Updated balance; transaction appended; or 'Invalid Amount' with deposit option
Transfer	Amount, Recipient name (String), Account number (int), Reference (String)	Confirmation screen; both sender and recipient history updated; or 'Account not found'
Register	Username, Full Name, Surname, Email, Phone, Address, Gender, Password, Account Type choice (1-3)	Generated account number, PIN, type confirmation; or 'Username already exists'
Close Account	Account number (int)	Status set to CLOSED; or 'Account is already closed' / 'Account Not Found'

4.11 Error Handling

Trust-Bank Scenario	How Our System Handled The Issue
Invalid login credentials	'Invalid Login Details!' displayed; user returned to main menu — no crash
Attempt to log in to a CLOSED account	'This account is closed. Please contact support.' — login blocked
Deposit amount ≤ 0	'Invalid Amount' displayed; user redirected to main menu
Withdraw amount > account balance	'Invalid Amount. Please deposit money...' with current balance shown; option to deposit
Transfer to non-existent account number	'Recipient account number not found. Transfer cancelled.' — returns to menu
Username already taken during registration	'Username already exists. Please choose another.' — registration cancelled
Closing an already-closed account	'Account is already closed.' — no duplicate status change
Invalid menu option entered	'Invalid option. Choose 1-N' displayed; menu re-displayed without crashing

5. USER MANUAL

For all non-technical user. Follow each section in order to set up and use the ***PLUTO BANKERS*** system.

5.1 Prerequisites

- **Java JDK 17 or later:** Download from https://www.oracle.com/java/technologies/downloads/
- **MySQL Server 8.0+:** Download from https://dev.mysql.com/downloads/mysql/
- **MySQL Workbench (optional):** For visual database management — https://dev.mysql.com/downloads/workbench/
- **NetBeans IDE (optional):** For running the Java project — https://netbeans.apache.org/

5.2How to Set Up the Database

1. Open MySQL Workbench (or any MySQL client).
2. Open a new SQL script tab.
3. Run the following scripts IN ORDER (order matters due to foreign key dependencies):

```
Step 1: Trust_Bank_DataBase_Design.sql     -- Creates the database
Step 2: clients_table_and_sample_data.sql  -- Creates clients table + inserts 5 clients
Step 3: employees_table_and_sample_data.sql -- Creates employees table + inserts 3 employees
Step 4: accounts_table_and_sample_data.sql  -- Creates accounts table + inserts 5 accounts
Step 5: transactions_table_and_sample_data.sql -- Creates transactions table + inserts 11 records
Step 6: transaction_history_table_and_sample_data.sql -- Creates history table + inserts 9 records
```

4. After running all scripts, verify with: SELECT * FROM clients; — you should see 5 rows.

5.3 How to Run the Application

Option A — Using NetBeans IDE

5. Open NetBeans and select File > Open Project.
6. Navigate to the project folder containing FinalizedBankingSystem.java and open it.
7. Right-click the project in the Projects panel and select Run.
8. The console window at the bottom will display: ===== WELCOME TO Trust-BANK =====

Option B — Using Command Line (Terminal)

```
# 1. Navigate to the folder containing FinalizedBankingSystem.java
cd path/to/your/project


# 2. Compile the Java file
```

```
javac FinalizedBankingSystem.java

# 3. Run the compiled class
java FinalizedBankingSystem
```

5.4 How to Use the System — Step by Step

Logging in as a Client

9. At the main menu, enter: 2 (Client Login)
10. Enter a username from the list below and press Enter:

Username	Password	Account Type	Balance
bob	bob1	Savings Account	R20,034.00
peter	peter1	Checking Account	R54,334.00
josh	josh1	Salary Account	R6,454.00
jack	jack1	Savings Account	R9,544.00

11. Note: 'sam' (CUS04) account is CLOSED — login will be blocked.

Performing a Deposit

12. From the client menu, enter: 2 (Deposit)
13. Enter: 1 to deposit into your account type
14. Enter the amount to deposit (must be greater than 0)
15. A confirmation is shown with the new balance
16. Enter 1 to return home or 2 to exit

Performing a Withdrawal

17. From the client menu, enter: 3 (Withdraw)
18. Enter the amount to withdraw (must not exceed available balance)
19. Enter: 1 to Proceed or 2 to Cancel
20. Success message and new balance are displayed

Transferring Money

21. From the client menu, enter: 4 (Transfer Money)
22. Enter the amount, the recipient's name, their account number, and a reference
23. The system validates the recipient account number — if not found, the transfer is cancelled
24. Review the confirmation screen and enter 1 to Proceed or 2 to Cancel

Logging in as an Employee

25. At the main menu, enter: 1 (Employee Login)
26. Use any of these credentials: mike/emp1, john/emp2, tumi/emp3
27. The employee menu will display with 6 options

5.5 Common Errors and Solutions

Error	Cause	Solution
'Invalid Login Details!'	Wrong username or password entered	Check the credentials table above; usernames and passwords are case-sensitive
'This account is closed. Please contact support.'	Attempting to log in with CUS04 (sam)	Use a different client account — CUS04 has been closed in the sample data
'Invalid Amount. Please deposit money...'	Entered withdrawal or transfer amount exceeds balance	First deposit funds; the system will show your current balance
'Recipient account number not found.'	Entered a non-existent account number in transfer	Valid account numbers are: 9343343, 4847433, 2987654, 3043433, 8493733
'Username already exists.'	Trying to register with a username already in the system	Choose a different username not already in use
Application won't start — 'java not found'	Java JDK not installed or not on system PATH	Install JDK 17+ and ensure JAVA_HOME environment variable is set

6. CHALLENGES AND REFLECTIONS

6.1 Challenges Faced During Development

Challenge 1: Balance Not Persisting After Transactions

One of the earliest bugs encountered whilst constructing this system was that deposits and withdrawals appeared to succeed (the new balance was printed on screen) but did not actually change the stored balance. From doing sub research, we later on found that because the code used local variables (double balanceM = 20000) inside each method instead of reading from and writing back to the accountBalance[] array.

Impact: Every time a method was called, it reset the balance to the hardcoded value, making it impossible to test transactions. A client could withdraw R20,000 and then withdraw another R20,000 in the same session.

Resolution: All balance operations were refactored to read from accountBalance[loggedInUser] at the start and write back using accountBalance[i] += amount or accountBalance[i] -= amount. The local hardcoded variables were completely removed.

Challenge 2: Duplicate Scanner Causing Input Errors

The transfer() method originally created a new Scanner object inside the method (Scanner s = new Scanner(System.in)). This conflicted with the static Scanner s already declared at the class level, causing the application to skip input prompts and throw NoSuchElementException errors which were a pain to deal with.

Impact: The transfer feature was completely unusable. Running it caused the application to crash or skip prompts, making it impossible to enter recipient details.

Resolution: The duplicate Scanner declaration inside transfer() was removed. The method now uses the shared static Scanner s. A s.nextLine() call was added after s.nextInt() to consume the leftover newline character in the input buffer.

Challenge 3: Transaction History Not Being Recorded

The transactionHistory[] array was declared but never written to — no method called String.format() to append a record. This meant the View Transaction History feature always displayed 'No Transactions Available'.

Resolution: Each transaction method (deposit, withdraw, transfer) was updated to append a formatted string to transactionHistory[i] immediately after updating the balance. Transfer also records an entry for the recipient's history index.

Challenge 4: Database Table Creation Order (Foreign Key Dependencies)

When the SQL scripts were initially run in alphabetical order, the accounts table creation failed because it references clients, which had not yet been created. Similarly, transactions failed because it references both accounts and employees.

Resolution: The correct execution order was established: clients first, then employees, then accounts, then transactions, then transaction_history. This order is documented in the User Manual (Section 5.2).

6.2 Lessons Learned

- **Always use shared resources:** Static variables shared across methods (Scanner, arrays) prevent resource conflicts. Declaring new instances inside methods of the same class is almost always wrong.
- **Test incrementally:** Running the application after implementing each feature (not all at once) makes bugs far easier to trace. Both the balance and Scanner bugs would have been caught immediately.
- **State management matters:** In a stateful application, every method that changes data must write back to the central data store — not calculate locally and discard.
- **SQL script order is not arbitrary:** Foreign key constraints mean table creation must follow a dependency-first order. Documenting this order saves time during setup and submission.

6.3 Individual Contribution Statement

Student Name	Student Number	Contribution
Shaun Khalira	MA977363	Java application structure, client login, deposit, withdrawal, transfer methods
CHACY FUNWI	1195270	Employee portal, client registration, view/edit/close client features
Shaun khalira && Funwi Checy Che	----------	Database design, SQL schema, all INSERT/SELECT/UPDATE/DELETE queries
Jordan Olivier		Documentation report, ERD design, user manual, testing and bug logging

APPENDIX A — FULL SQL SCHEMA & QUERIES

A.1 Database Creation

Pluto_Bankers DataBase Design.sql

```sql
CREATE DATABASE Pluto_Banking_System;

USE Pluto_Banking_System;

```

A.2 clients Table

clients table and sample data.sql

```sql
CREATE TABLE clients (
    client_id Varchar(50) PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(100) NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    surname VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    phone VARCHAR(20),
    address TEXT,
    gender VARCHAR(10),
    status VARCHAR(20) DEFAULT 'ACTIVE'
);

INSERT INTO clients (client_id, username, password, full_name, surname, email, phone, address, gender, status)
VALUES
('CUS01', 'bob', 'bob1', 'John Mark', 'Mark', 'john@gmail.com', '0673459873', '21 mooi street JHB', 'Male', 'ACTIVE'),
('CUS02', 'peter', 'peter1', 'Paul Mcman', 'Mcman', 'paul@gmail.com', '0663452363', '101 good street PTA', 'Male', 'ACTIVE'),
('CUS03', 'josh', 'josh1', 'Sandra Moore', 'Moore','sandra@gmail.com', '0654231200','211 sandom street JHB', 'Female', 'ACTIVE'),
('CUS04', 'sam', 'sam1', 'Peter Zulu', 'Zulu','peter@gmail.com', '0697653473','301 nitye street JHB', 'Male', 'CLOSED'),
('CUS05', 'jack', 'jack1', 'Hazard Disman', 'Disman','hazard@gmail.com', '0634568765','91 mooi street CPT', 'Male', 'ACTIVE'),
;

```

A.3 employees Table

employees table and sample data.sql

```sql
CREATE TABLE employees (
    employee_id varchar(50) PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(100) NOT NULL,
    full_name VARCHAR(100)
);

INSERT INTO employees(employee_id, username, password, full_name)
VALUES
('EMP01', 'mike', 'emp1', 'Mike Smith'),
('EMP02', 'john', 'emp2', 'John Peterson'),
('EMP03', 'tumi', 'emp3', 'Tumi Daniels');
```

A.4 accounts Table

accounts table and sample data.sql

```sql
CREATE TABLE accounts (
    account_id INT AUTO_INCREMENT PRIMARY KEY,
    client_id VARCHAR(50) NOT NULL,
    account_number BIGINT UNIQUE NOT NULL,
    account_pin INT NOT NULL,
    account_type VARCHAR(50),
    balance DECIMAL(12,2) DEFAULT 0.00,
    FOREIGN KEY (client_id) REFERENCES clients(client_id)
);

INSERT INTO accounts (account_id, client_id, account_number, account_pin, account_type, balance)
VALUES

(1, 'CUS01', 9343343, 1234, 'Savings Account', 20034.00),
(2, 'CUS02', 4847433, 2345, 'Checking Account', 54334.00),
(3, 'CUS03', 2987654, 3456, 'Salary Account', 6454.00),
(4, 'CUS04', 3043433, 4567, 'Joint Account', 8445.00),
(5, 'CUS05', 8493733, 5678, 'Savings Account', 9544.00),


```

A.5 transactions Table

transactions table and sample data.sql

```sql
CREATE TABLE transactions (
    transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    account_id INT NOT NULL,
    employee_id varchar(50),
    client_id Varchar(50),
    transaction_type VARCHAR(50),
    amount DECIMAL(12,2),
    reference_note VARCHAR(255),
    transaction_date date DEFAULT CURRENT_DATE(),
    FOREIGN KEY (account_id) REFERENCES accounts(account_id),
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id),
    FOREIGN KEY (client_id) REFERENCES clients(client_id),
    Time tIME DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO transactions (account_id, employee_id, client_id, transaction_type, amount, reference_note)
VALUES


(1, 'EMP01', 'CUS01', 'DEPOSIT', 5000.00, 'Cash Deposit'),
(1, 'EMP02', 'CUS01', 'WITHDRAWAL', 1200.00, 'ATM Withdrawal'),
(1, 'EMP03', 'CUS01', 'TRANSFER SENT', 2500.00, 'Transfer to Peter'),

(2, 'EMP01', 'CUS02', 'DEPOSIT', 10000.00, 'Salary Payment'),
(2, 'EMP02', 'CUS02', 'TRANSFER RECEIVED', 2500.00, 'Received from Bob'),
(2, 'EMP03', 'CUS02', 'WITHDRAWAL', 3000.00, 'ATM Withdrawal'),

(3, 'EMP01', 'CUS03', 'DEPOSIT', 4500.00, 'Monthly Salary'),
(3, 'EMP02', 'CUS03', 'TRANSFER SENT', 1500.00, 'Transfer to Linda'),

(4, 'EMP03', 'CUS04', 'WITHDRAWAL', 500.00, 'Store Purchase'),

(5, 'EMP01', 'CUS05', 'DEPOSIT', 2000.00, 'Cash Deposit'),
(5, 'EMP02', 'CUS05', 'TRANSFER RECEIVED', 800.00, 'Received from Kevin'),

```

A.6 transaction_history Table

transaction_history table and sample data.sql

```sql

CREATE TABLE transaction_history (
    history_id INT AUTO_INCREMENT PRIMARY KEY,
    account_number BIGINT,
    activity TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (account_number) REFERENCES accounts(account_number),
    employee_id varchar(50),
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id)
);

INSERT INTO transaction_history (account_number, activity, employee_id)
VALUES

(9343343, 'DEPOSIT | Amount: R5000.00 | New Balance: R25034.00', 'EMP01'),
(9343343, 'WITHDRAWAL | Amount: R1200.00 | New Balance: R23834.00', 'EMP02'),
(9343343, 'TRANSFER SENT | To: Peter (Acc: 8493733) | Amount: R2500.00 | Ref: Transfer to Peter | N

(4847433, 'DEPOSIT | Amount: R10000.00 | New Balance: R64334.00', 'EMP01'),
(4847433, 'TRANSFER RECEIVED | From: Bob (Acc: 8493733) | Amount: R2500.00 | Ref: Received from Bob
(4847433, 'WITHDRAWAL | Amount: R3000.00 | New Balance: R63834.00', 'EMP03'),

(2987654, 'DEPOSIT | Amount: R4500.00 | New Balance: R6454.00', 'EMP01'),
(2987654, 'TRANSFER SENT | To: Linda (Acc: 3043433) | Amount: R1500.00 | Ref: Transfer to Linda | N

(3043433, 'WITHDRAWAL | Amount: R500.00 | New Balance: R7945.00', 'EMP03'),


```

A.7 SELECT Queries

select1.sql

```sql
select * from employees
order by username desc;
```

select2.sql

```sql
select * from clients
order by full_name asc;
```

select3.sql

```sql
select full_name from clients
where client_id = 'CUS01';
```

select4.sql

```sql
select full_name from employees
where employee_id = 'EMP01';
```

select5.sql

```sql
select Full_name from clients join transactions on clients.client_id = transactions.client_id
where transactions.transaction_type = 'Deposit' and transactions.amount > 1000;
```

A.8 UPDATE Queries

update1.sql

```sql
update clients
set full_name = 'John Doe' where client_id = 'CUS01';
```

update2.sql

```sql
update employees
set full_name = 'Michael Smith' where employee_id = 'EMP01';
```

A.9 DELETE Queries

delete1.sql

```sql
delete from employees
where employee_id = 'EMP01';
```

delete2.sql

```sql
delete from clients
where client_id = 'CUS01';
```

JAVA SOURCE CODE REFERENCE

Class Structure

Method	Access	Purpose
main(String[] args)	public static	Entry point; calls mainMenu()
mainMenu()	static	Displays main menu; routes to employee login, client login, register, or exit
clientLogin()	static	Validates client credentials; checks ACTIVE status; routes to menu2()
employeeLogin()	static	Validates employee credentials; sets loggedInEmployee; routes to employeeMenu()
registerClient()	static	Collects registration data; validates username uniqueness; appends to all arrays
menu2()	static	Client main menu loop; handles all 8 client options
check_balance()	static	Displays balance from accountBalance[]; routes to sub-actions
deposit()	static	Adds amount to accountBalance[]; records to transactionHistory[]
withdraw()	static	Subtracts from accountBalance[] after balance validation; records history
transfer()	static	Validates recipient; deducts from sender; credits recipient; records both histories
updateDetails()	static	Updates password, address, email, or phone in the relevant array
viewTransactionHistory(int)	static	Prints transactionHistory[customerIndex] or 'No Transactions Available'
employeeMenu()	static	Employee menu loop; handles all 6 employee options
viewClients()	static	Iterates all client arrays; prints formatted record for each
viewClientProfile()	static	Looks up client by account number; prints full profile
editClient()	static	Searches by account number; allows editing 4 fields
closeAccount()	static	Sets status[i] = 'CLOSED'; logs acting employee
appendString(String[], String)	static	Grows a String array by 1; used in registerClient()
appendInt(int[], int)	static	Grows an int array by 1; used in registerClient()
appendDouble(double[], double)	static	Grows a double array by 1; used in registerClient()

Data Arrays — All Static Fields

Array	Type	Size	Contents
username	String[]	5 (+ dynamic)	Login usernames for all clients
password	String[]	5 (+ dynamic)	Login passwords for all clients
fullName	String[]	5 (+ dynamic)	Full names of all clients
surname	String[]	5 (+ dynamic)	Surnames of all clients
email	String[]	5 (+ dynamic)	Email addresses
phone	String[]	5 (+ dynamic)	Phone numbers
address	String[]	5 (+ dynamic)	Residential addresses
gender	String[]	5 (+ dynamic)	Gender values
accountType	String[]	5 (+ dynamic)	Account type for each client
status	String[]	5 (+ dynamic)	ACTIVE or CLOSED per client
clientID	String[]	5 (+ dynamic)	Client IDs (CUS01–CUS05)
accountNumber	int[]	5 (+ dynamic)	7-digit account numbers
accountPin	int[]	5 (+ dynamic)	4-digit PINs
accountBalance	double[]	5 (+ dynamic)	Current balance per client
transactionHistory	String[]	5 (+ dynamic)	Formatted session transaction log per client
employeeUsername	String[]	3 (fixed)	Employee login usernames
employeePassword	String[]	3 (fixed)	Employee login passwords